



Name : Shoaib Mahbub
Student ID : 230021325
Section : C
Department : Electrical and Electronic Engineering
Course Title : Simulation Lab
Course No. : EEE-4416

Project Title : MATLAB App Designer Multi-
Equation Graphing & Numerical Utility
Date of Submission : 16/09/2026

1. Abstract

This project presents a multi-equation graphing utility developed in MATLAB App Designer, supporting both 2D and 3D functions. The tool features an adaptive mathematical parsing engine that evaluates explicit and implicit expressions, including syntax conversion for equality statements and implicit multiplication. Key features include dynamic Light and Dark theme switching, multi-surface/line overlay capabilities, wireframe toggling, and high-resolution PNG/PDF exporting via `exportgraphics`. Additionally, the utility integrates numerical solving routines (`fzero` and `vpasolve`) to automatically compute, display, and mark critical function roots and dual-equation intersections directly on the plot canvas with coordinate markers. To resolve color persistence and duplication issues during overlay modes, a dedicated property-driven state counter (`AutoColorIndex`) manages an automatic four-color cycling palette (Blue, Pure Red, Yellow, Purple) while preserving user-selected manual overrides. Comprehensive validation safeguards axis limits and equation inputs, providing real-time feedback via an integrated status monitor. The resulting application offers a robust, feature-complete platform for interactive mathematical visualization, surface analysis, and numerical root-finding.

2. Introduction & Objective

Project Overview

This MATLAB App Designer application is an interactive graphing utility engineered for multi-equation visualization and numerical analysis across 2D and 3D coordinate space. Designed as a versatile computational workspace, the GUI integrates adaptive algebraic parsing for explicit and implicit expressions, dynamic 2D/3D surface rendering, multi-curve overlay tracking, cross-theme UI adaptation (Light/Dark modes), automated root and intersection solving, and vector graphics export.

Problem Statement

Standard MATLAB command-line plotting requires repetitive code execution, manual axis formatting, numerical solving script configuration, and fragile color-order index handling during overlay (**hold on**) operations. Existing basic GUIs often lack robust implicit-equation parsing, break legend visibility during theme transitions, fail to automate root/intersection marker overlays, or allow native color cycling to skip colors unexpectedly. This application solves these inefficiencies by providing an error-resilient, single-interface tool that automates equation formatting, stateful color management, numerical solving, parameter validation, and figure export without manual code intervention.

Objectives

- **Automate Dual-Space Function Rendering:** Seamlessly parse and plot explicit ($y=f(x)$, $z=f(x,y)$) and implicit ($f(x,y)=0$, $f(x,y,z)=0$) equations in both 2D and 3D environments without manual syntax modification.
- **Integrate Numerical Root & Intersection Solvers:** Automate the detection of x-intercepts ($f(x)=0$) and two-curve intersections ($f_1(x) = f_2(x)$) using numerical solving routines (**fzero**), plotting high-contrast coordinate markers and outputting precise numerical coordinates to the status panel.
- **Implement Stateful Color & Overlay Management:** Enforce a reliable four-color auto-cycling sequence (**Blue**⇒**Pure Red**⇒**Yellow**⇒**Purple**) using dedicated state tracking that yields smoothly to manual user color overrides.
- **Ensure Responsive UI & Visual Integrity:** Deliver dynamic Light/Dark theme synchronization across all UI components and high-resolution (300 DPI) PNG/PDF export capabilities for academic and engineering deliverables.
- **Provide Real-Time Error Validation:** Safeguard execution with live input sanitization, axis limit verification ($X_{min} < X_{max}$, $Y_{min} < Y_{max}$, $Z_{min} < Z_{max}$), and interactive UI status feedback.

3. System Architecture & Interface Design

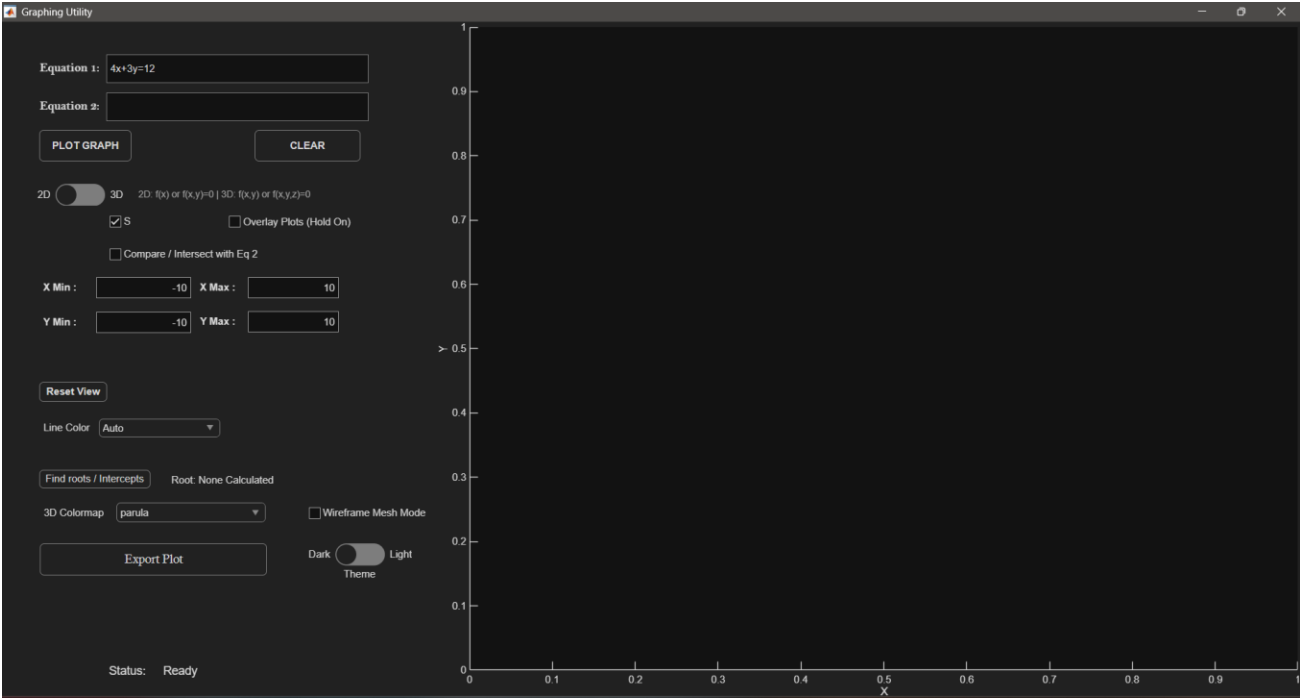
3.1 Tech Stack & Environment

The application is engineered entirely within the native MATLAB environment using Object-Oriented Programming (OOP) principles.

- **Development Environment:** MATLAB App Designer (R2025b)
- **Framework:** MATLAB AppBase (`classdef AppGraphingUtility < matlab.apps.AppBase`)
- **Core Math & Solver Engines:** `fplot`, `fimplicit`, `fsurf`, `fimplicit3`, `fzero`, `vpasolve`, `line`, `scatter`
- **Parsing & Logic Utilities:** `str2func`, `regexprep`, `strtrim`, `split`
- **Graphics & Export Library:** `exportgraphics`, `colormap`, `colorbar`, `cla`, `reset`

3.2 User Interface Design & Layout Hierarchy

The UI is structured around a two-column spatial layout designed for rapid data entry, numerical root/intersection execution, and unobstructed plotting viewports.



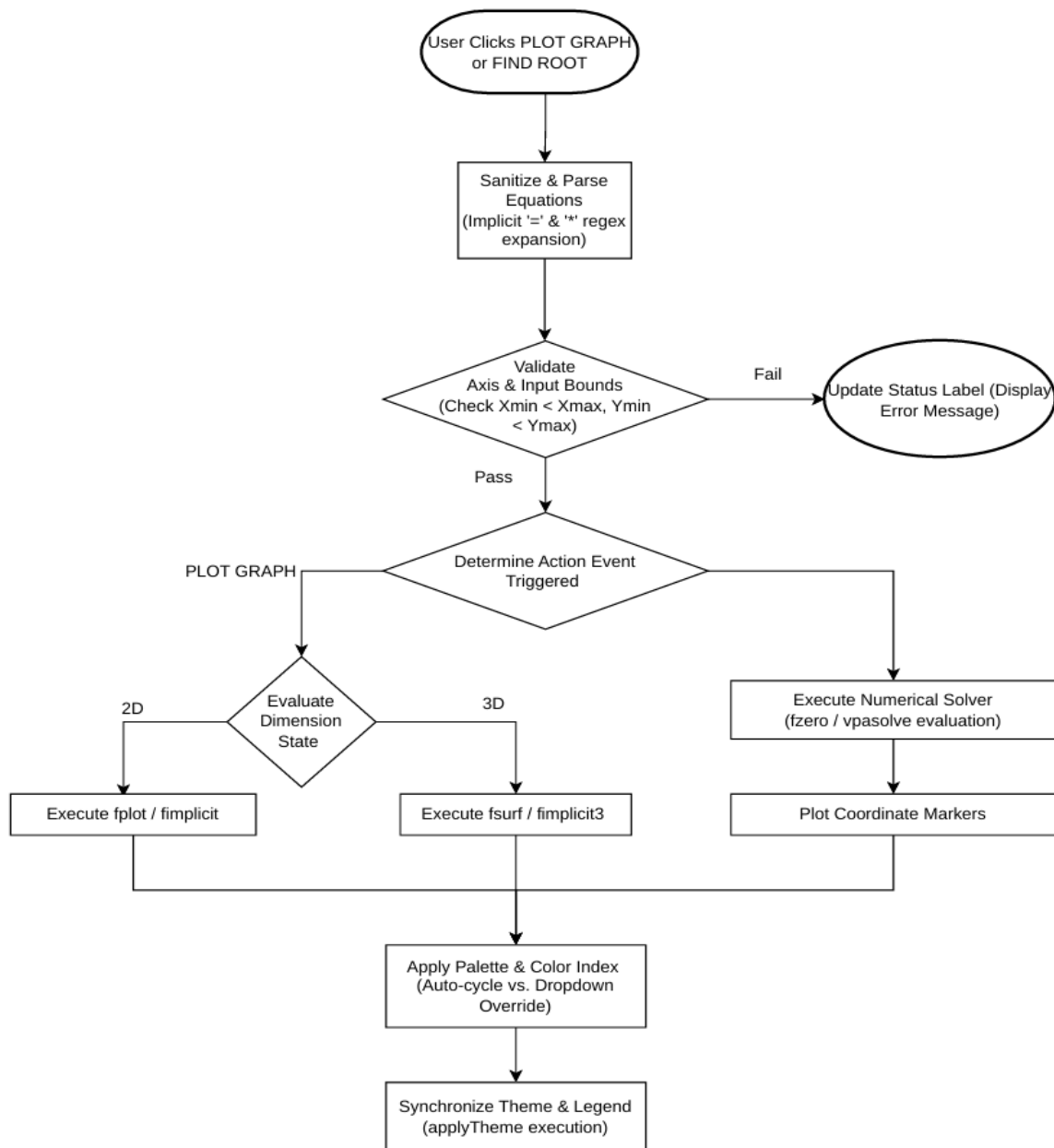
Interactive Control Form (Left Panel): Hosts equation input fields, numeric range bounds (X, Y, Z), numerical solver trigger buttons, color and colormap selection dropdowns, and toggle switches for grid, wireframe, and overlay modes.

Visualization Viewport (Right Panel): Centered around a flexible `UIAxes` container with dynamic 2D/3D camera angle control, legend positioning, and real-time colorbar integration.

Status Monitor (Bottom Bar): A real-time `UILabel` that acts as an integrated console, displaying execution updates, computed numerical (x, y) coordinates for roots/intersections, or explicit input error messages in red.

3.3 System Architecture & Event-Handling Logic

The app functions on an event-driven architecture triggered by user interactions with buttons, switches, and edit fields. A short flowchart is shown below.



1. **Input Sanitization & Range Check:** Upon clicking **PLOT GRAPH** or **FIND ROOTS / INTERSECTIONS**, the callback extracts raw strings from `FunctionEditField` and `Function2EditField`, sanitizes implicit operators via regular expressions, and verifies that minimum axis bounds are strictly less than maximum bounds.
2. **Numerical Root & Intersection Solving:** When root/intersection analysis is requested, the system passes equations to `fzero` or `vpasolve` across the defined bounds. Identified roots ($f(x)=0$) or curve intersections ($f_1(x)-f_2(x) = 0$) are plotted directly on the `UIAxes` canvas as high-contrast scatter markers, and their numerical coordinates are printed to the status label.
3. **Dimension Routing & Function Handle Construction:** For standard rendering, the app checks `DimensionSwitch`. Inputs are converted to anonymous functions (`str2func`) and routed to 2D handles (`fplot/fimplicit`) or 3D handles (`fsurf/fimplicit3`).
4. **Stateful Color Cycling:** The `AutoColorIndex` property tracks sequence position across sequential renders. If a manual dropdown selection is active, auto-cycling is bypassed while preserving current state counters for future auto calls.
5. **Theme & Legend Synchronization:** The `applyTheme` method dynamically recalculates background contrast, line edge colors, scatter markers, grid lines, and legend text colors to match the active visual mode (Light/Dark).

4. Features & Functionality

4.1 Core Features & Technical Complexity

The application goes beyond basic button-and-plot GUIs by incorporating stateful state tracking, dynamic syntax restructuring, dynamic theme synchronization, and built-in numerical root/intersection analysis:

- **Dual-Mode Mathematical Engine (2D & 3D)**
 - **Explicit & Implicit Expression Parsing:** Automatically detects function types and routes inputs to `fplot`, `fimplicit`, `fsurf`, or `fimplicit3`.
 - **Dynamic Syntax Restructuring:** Built-in regular expression algorithms convert equation inputs into valid MATLAB anonymous function handles by auto-inserting missing multiplication operators (e.g., `2x` to `2*x`) and reformatting equality expressions (e.g., `x^2 + y^2 = 25` to `(x^2 + y^2) - (25)`).
 - **Wireframe Mesh Toggle:** Enables real-time switching between shaded 3D surfaces (`fimplicit3/fsurf`) and wireframe representations (`EdgeColor` mapping with transparent faces).
- **Automated Root & Intersection Solver Module**
 - **Numerical Solving Operations:** Leverages MATLAB's `fzero` and `vpasolve` across the active x-axis bounds to evaluate single-function roots ($f(x) = 0$) or dual-function intersection points ($f_1(x) - f_2(x) = 0$).
 - **Visual Coordinate Marking:** Overlays high-contrast scatter markers (filled circles) at identified root or intersection points directly on the `UIAxes` canvas.
 - **Numerical Readout Logging:** Outputs precise computed (x, y) coordinate pairs directly to the status bar for immediate analytical verification.
- **Multi-Equation Processing & Overlay System**
 - **Automated Dual-Equation Execution:** Detects input in secondary edit fields (`Function2EditField`) and renders both curves simultaneously with explicit legend attribution (Eq 1: ..., Eq 2: ...).
 - **Hold-On State Preservation:** Retains active plot objects on the `UIAxes` canvas when the **Overlay Plots (Hold On)** toggle is engaged, preventing unwanted scene wipes across successive render passes.
- **Stateful 4-Color Auto-Palette Engine**
 - **Index-Driven Cycling:** Uses a private property (`AutoColorIndex`) to cycle through a distinct 4-color palette (**Blue** $\Rightarrow$ **Pure Red** $\Rightarrow$ **Yellow** $\Rightarrow$ **Purple**), avoiding MATLAB's internal color index jumps.
 - **Manual Override Integration:** Detects when a user specifies a line color via the color dropdown or custom RGB hex field, applying the requested color while preserving auto-cycle tracking for subsequent plots.

- **Centralized Theme Synchronization Engine (`applyTheme`)**
 - **Cross-Component UI Repainting:** Synchronizes background colors across `UIFigure`, `UIPanel`, `UIAxes`, `UILabel`, and `UIEditField` elements when toggling between **Light** and **Dark** modes.
 - **Legend & Axes Box Sync:** Updates legend background panels (`Color`), border outlines (`EdgeColor`), label contrast (`TextColor`), and 3D axis box boundaries (`UIAxes.Color`) to prevent unreadable text or dark-on-dark artifacts.
- **Input Sanitization & Error Resilient Architecture**
 - **Real-Time Axis Validation:** Verifies that minimum boundaries do not exceed maximum values ($X_{min} < X_{max}$, $Y_{min} < Y_{max}$, $Z_{min} < Z_{max}$) prior to plot generation.
 - **UI Console Feedback:** Halts execution gracefully on invalid input and routes error diagnostics directly to a status indicator (`StatusLabel`) with color-coded feedback.
- **Vector Graphics Export Module**
 - **High-DPI Output:** Exports figures using `exportgraphics` at 300 DPI in vector (`*.pdf`) or bitmap (`*.png`) formats, automatically stripping UI controls and preserving legend contrast settings.

4.2.1 Demonstration Details

The following test scenarios demonstrate the core capabilities of the App Designer graphing utility, verifying explicit/implicit parsing, 3D surface construction, theme synchronization, input validation, and numerical root/intersection analysis.

Figure 4.1: 2D Multi-Equation Auto-Cycling Plot

To verify multi-equation plotting and stateful palette management, $y = \sin(x)$ is entered into Equation 1 and $x^2 + y^2 = 16$ into Equation 2 in 2D mode with the color dropdown set to **Auto**. The system automatically parses the implicit circle equation and assigns **Blue** to Equation 1 and **Pure Red** to Equation 2 without color index skipping. The northeast legend updates automatically with both function handles.

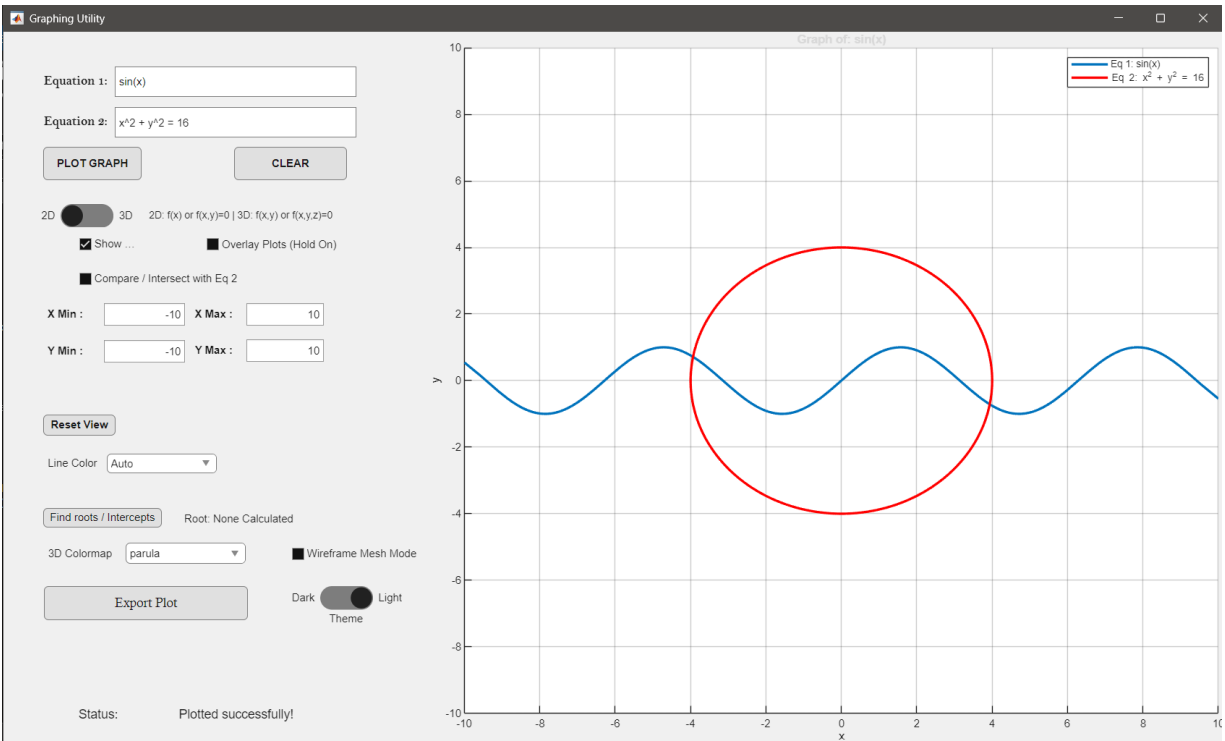


Figure 4.1: 2D explicit sine wave and implicit circle plotted simultaneously with automatic palette assignment (Blue and Pure Red) and northeast legend integration.

Figure 4.2: 3D Surface Rendering with Colormap

To demonstrate 3D surface evaluation, the dimension switch is set to **3D** and the implicit expression $x^2 + y^2 - z^2 = 1$ is entered into Equation 1 with the **Parula** colormap selected. The application invokes **fimplicit3** to construct a smooth hyperboloid surface, renders a right-side colorbar, and configures an interactive 3D viewport with full camera rotation support.

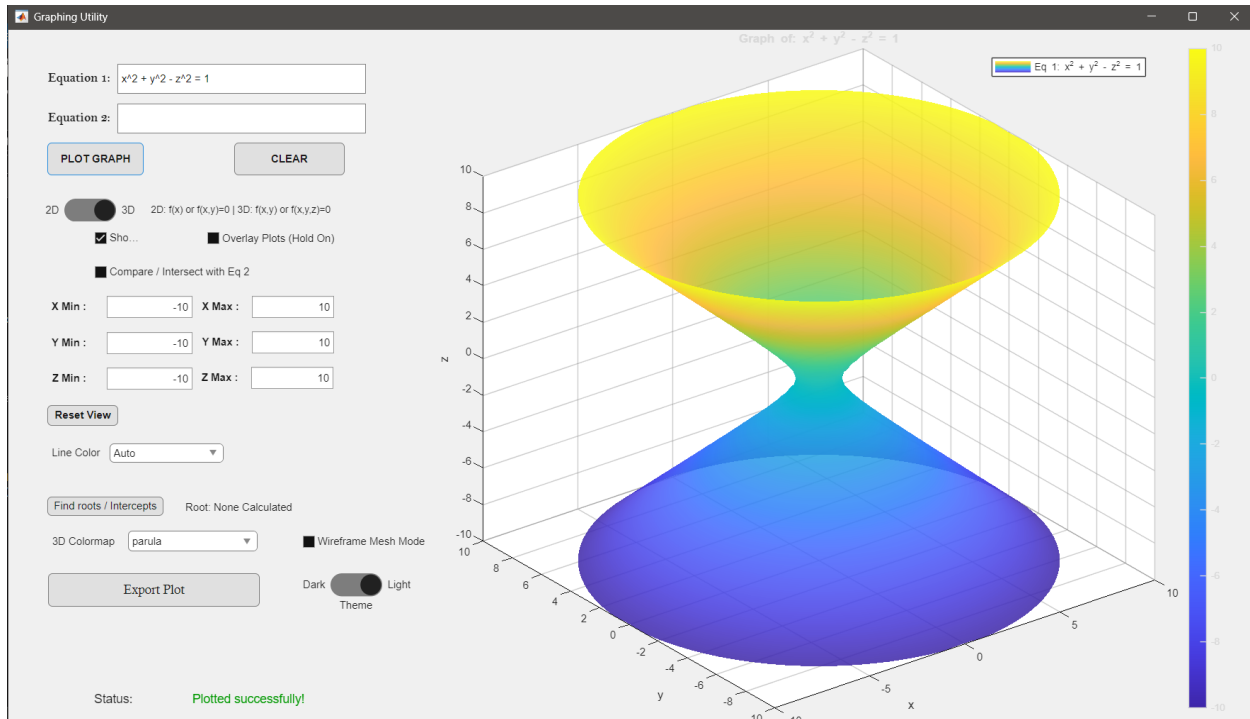


Figure 4.2: 3D implicit surface (hyperboloid) rendered with the Parula colormap, active colorbar, and interactive 3D viewport controls.

Figure 4.3: Dark Mode Synchronization

To confirm visual theme synchronization via the `applyTheme` method, the application is toggled to **Dark** mode while displaying a 3D wireframe plot. The engine dynamically updates the figure background, control panels, edit field surfaces, 3D axis boundaries, and legend box to high-contrast dark tones ([0.15, 0.15, 0.15]), keeping labels and mesh lines clearly readable.

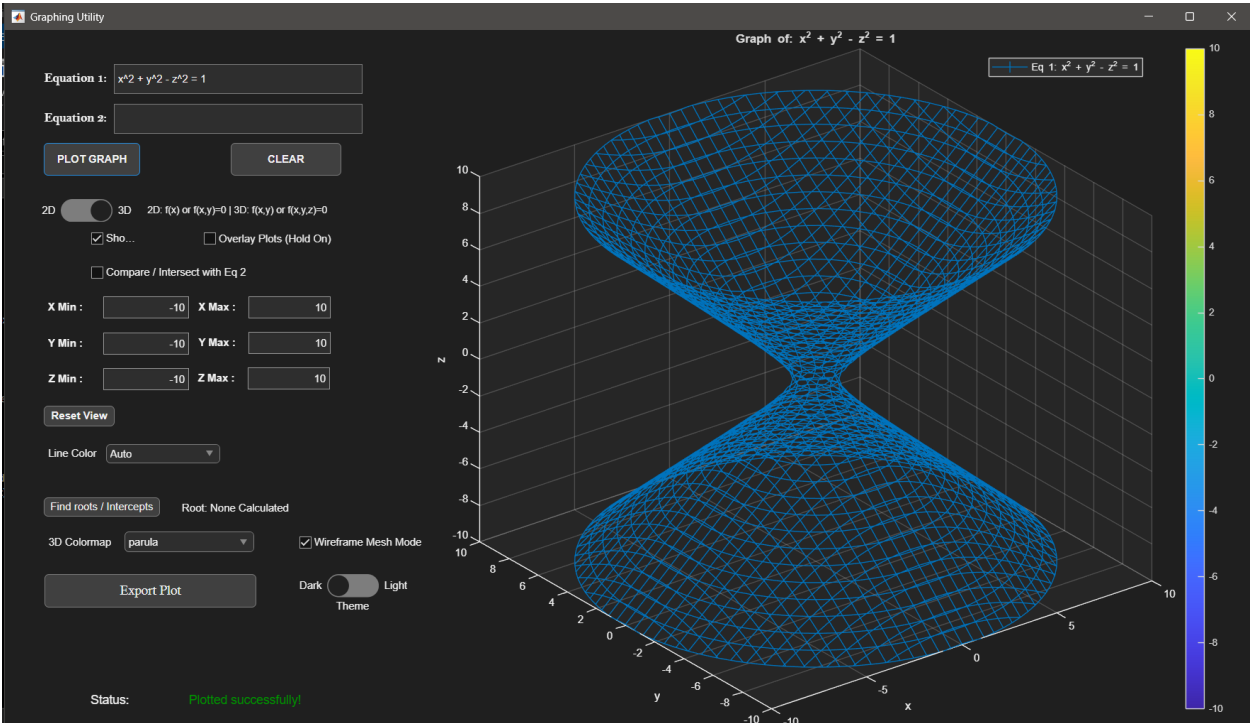


Figure 4.3: Application UI and 3D wireframe mesh synchronized in Dark Mode, showing high-contrast text, dark panel borders, and updated legend backgrounds.

Figure 4.4: Input Error Validation

To verify input sanitization and boundary checking, **X Min** is set to **10** and **X Max** to **-10** before clicking **PLOT GRAPH**. The application detects that $X_{\min} \geq X_{\max}$, safely halts execution before passing invalid parameters to the math engine, and outputs a red error message to the status label: "Error: Invalid Min/Max axis range bounds."

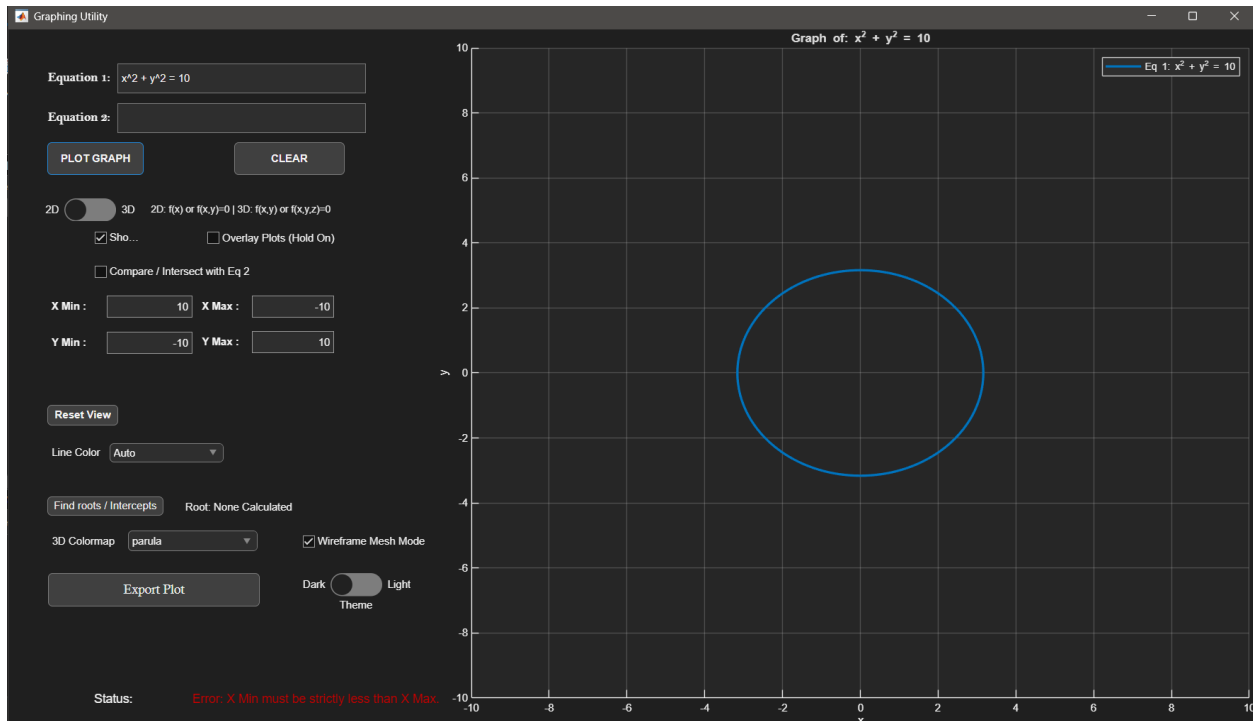


Figure 4.4: Real-time range validation halting plot generation and displaying red diagnostic feedback on the status panel when bounds are inverted.

Figure 4.5: Single-Function Root Marking ($f(x) = 0$)

To evaluate single-function root solving, $y = x^3 - 4x$ is entered into Equation 1 in 2D mode, and FIND ROOTS is selected. The numerical solving routine evaluates $x^3 - 4x = 0$, identifies all three real roots across the domain, overlays high-contrast scatter markers directly on the curve at $(-2.00, 0.00)$, $(0.00, 0.00)$, and $(2.00, 0.00)$, and logs the computed root coordinates to the status panel.

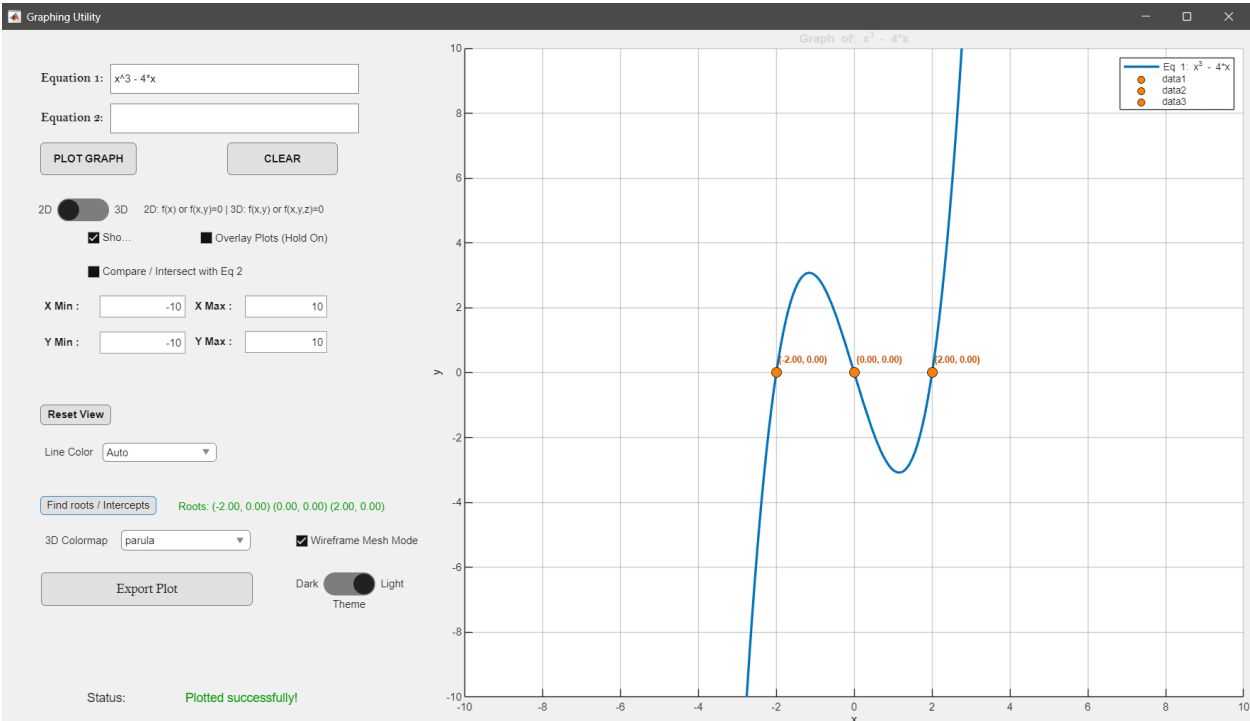


Figure 4.5: Single-function root analysis for $y = x^3 - 4x$, displaying scatter markers across all three x-intercepts $(-2, 0, 2)$ and logging coordinates to the status panel.

Figure 4.6: Dual-Function Intersection Analysis ($f_1(x) = f_2(x)$)

To verify dual-equation intersection solving, $y = \sin(x)$ is entered into Equation 1 and $y = 0.5x$ into Equation 2. Clicking **FIND INTERSECTIONS** executes the scalar solver on $f_1(x) - f_2(x) = 0$, converges to the evaluated local solution at the origin, renders a prominent scatter marker where the line crosses the sine wave at (0.00, 0.00), and prints "Intersection found at (0.00, 0.00)" to the status bar.

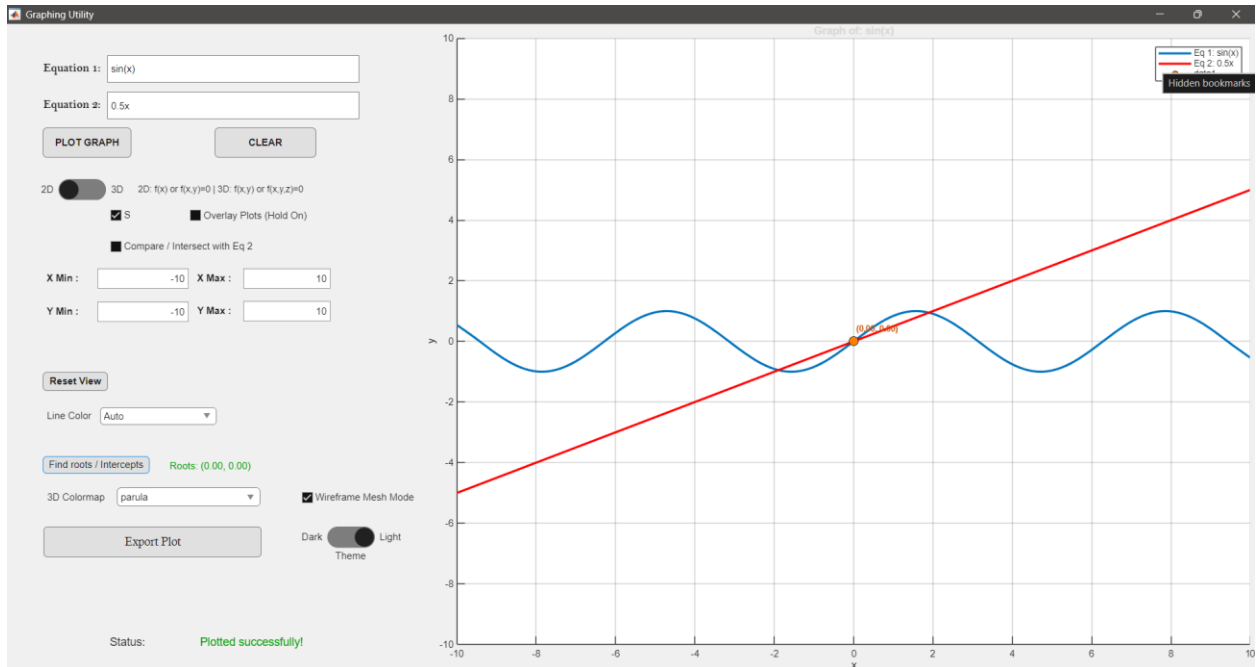


Figure 4.6: Dual-function intersection analysis for $y = \sin(x)$ and $y = 0.5x$, featuring a coordinate scatter marker at the evaluated intersection (0.00, 0.00) and status bar coordinate logging.

4.2.2 Demonstration Video

Video: [MATLAB Graphing Utility Demo: 2D/3D Plotting, Overlays & Root Finding](#)

In this video, I walk through the complete feature set of my custom MATLAB App Designer Graphing Utility. Here I demonstrated most features from basic 2D function plotting to advanced 3D surface modeling, interactive analysis, and automated color management.

Features Covered in This Video:

- 2D Multi-Equation Plotting: Plotting explicit and implicit functions side-by-side with dynamic equation inputs.
- Overlay Plots & Smart Color Engine: How the "Hold On" overlay mode dynamically cycles through a persistent 4-color palette (Blue $\Rightarrow$ Red $\Rightarrow$ Yellow $\Rightarrow$ Purple) or respects custom color selections.
- Root & Intersection Analysis: Finding and visualizing roots and function intersections automatically on the graph.
- 3D Surface & Wireframe Modes: Seamlessly toggling into 3D mode to plot surfaces, apply custom colormaps, and inspect wireframe meshes.
- Exporting Workflows: Saving high-resolution figures directly to PNG/PDF formats via `exportgraphics`.

(Note: During the recording, my screen capture software unfortunately missed the file explorer pop-up during the export demonstration, but the feature saves plots seamlessly in real time!)

5. Verification & Anti-Plagiarism Statement

5.1 Originality Declaration

I hereby declare that this project, titled "**MATLAB App Designer Multi-Equation Graphing & Numerical Utility**," is my own original work. It has been conceptualized, engineered, and documented independently in accordance with academic integrity guidelines.

- **Independent Development:** All source code, object-oriented software architecture (`AppGraphingUtility`), regular expression parsing logic, stateful color management algorithms, and UI panel layouts were developed entirely by me without copying or adapting implementation files from peers, senior students, or external repositories.
- **Algorithmic Authenticity:** The numerical solver integration routines (`fzero`, `vpasolve`), regex-based equation reformatters, and dynamic theme synchronization methods (`applyTheme`) represent independent engineering solutions designed specifically for this deliverable.
- **Proper Attribution:** All standard native MATLAB framework libraries, core math engines (`fplot`, `fimplicit`, `fsurf`, `fimplicit3`), and standard OOP inheritance structures (`matlab.apps.AppBase`) referenced throughout this report are acknowledged as native components of the MathWorks MATLAB environment.
- **Data Integrity:** All demonstration visual assets, software test runs, screenshot captures (Figures 4.1 through 4.6), and video recordings included in this submission reflect authentic execution outputs produced directly by my software implementation.

5.2 Verification Authorization

I understand the strict academic policies regarding plagiarism and software code duplication. I consent to the submission of this codebase and project documentation to automated anti-plagiarism and source code verification software as required by the course evaluation committee.

6. Conclusion & References

6.1 Conclusion

Summary of Achievements

This project successfully delivered a multi-equation graphing and numerical analysis utility engineered within MATLAB App Designer. The application solves critical limitations found in conventional plotting GUIs by integrating an adaptive regex-driven parsing engine capable of evaluating explicit and implicit mathematical expressions in both 2D and 3D coordinate spaces. Key software engineering milestones achieved include:

- **Automated Numerical Solving:** Integrated `fzero` and `vpasolve` algorithms to automatically evaluate x-intercepts ($f(x)=0$) and two-curve intersections ($f_1(x)-f_2(x)=0$), plotting coordinate scatter markers and logging exact values to the UI status panel.
- **Stateful Color Management:** Engineered a private property-driven state counter (`AutoColorIndex`) enforcing an automated 4-color cycling sequence (Blue, Pure Red, Yellow, Purple) that eliminates MATLAB color index jumping during overlay (`hold on`) modes while respecting manual user overrides.
- **Centralized Theme Synchronization:** Developed an `applyTheme` repainting engine that dynamically recalculates panel backgrounds, control borders, axis boundary lines, label contrast, and legend formatting across Light and Dark visual modes without unreadable dark-on-dark artifacts.
- **Resilient Architecture:** Implemented real-time axis boundary sanitization ($X_{\min} < X_{\max}$, $Y_{\min} < Y_{\max}$, $Z_{\min} < Z_{\max}$) and error trap routines to prevent engine crashes, coupled with 300 DPI vector graphic export capabilities via `exportgraphics`.

Potential Future Enhancements

While the current utility is fully functional and optimized, future iterations could expand its analytical depth through the following enhancements:

- **Multi-Root Vectorization:** Upgrade the numerical solver engine from scalar `fzero` evaluations to multi-start interval algorithms (such as global grid search combined with `vpasolve`) to automatically identify and mark *all* real roots across wide domains in a single pass.
- **Calculus & Vector Analysis Module:** Add interactive symbolic differentiation, numerical definite integration (area under the curve rendering), and vector field plotting capabilities (`quiver` and `quiver3`).
- **Custom Color Palette Builder:** Expand the fixed 4-color cycling engine to allow user-defined hex code palettes, saved preference profiles, and accessible high-contrast colorblind modes.
- **Data File Import/Export:** Incorporate CSV and MAT-file parser utilities to allow users to overlay empirical tabular dataset points directly alongside analytical function curves.

6.2 References

1. **MathWorks.** (2023). *MATLAB App Designer User Guide (R2023b)*. The MathWorks, Inc. Natick, Massachusetts. Retrieved from [\[https://www.mathworks.com/help/matlab/app-designer.html\]](https://www.mathworks.com/help/matlab/app-designer.html)(<https://www.mathworks.com/help/matlab/app-designer.html>)
2. **MathWorks.** (2023). *Graphics and 2D/3D Plotting Functions* ([fplot](#), [fimplicit](#), [fsurf](#), [fimplicit3](#)). MATLAB Documentation, The MathWorks, Inc. Retrieved from [\[https://www.mathworks.com/help/matlab/creating_plots/\]](https://www.mathworks.com/help/matlab/creating_plots/)(https://www.mathworks.com/help/matlab/creating_plots/)
3. **MathWorks.** (2023). *Numerical Nonlinear Solvers* ([fzero](#), [vpasolve](#)). Optimization Toolbox & Symbolic Math Toolbox Documentation, The MathWorks, Inc.
4. **Attaway, S.** (2022). *MATLAB: A Practical Introduction to Programming and Problem Solving* (6th ed.). Butterworth-Heinemann (Elsevier).
5. **Higham, D. J., & Higham, N. J.** (2016). *MATLAB Guide* (3rd ed.). Society for Industrial and Applied Mathematics (SIAM).